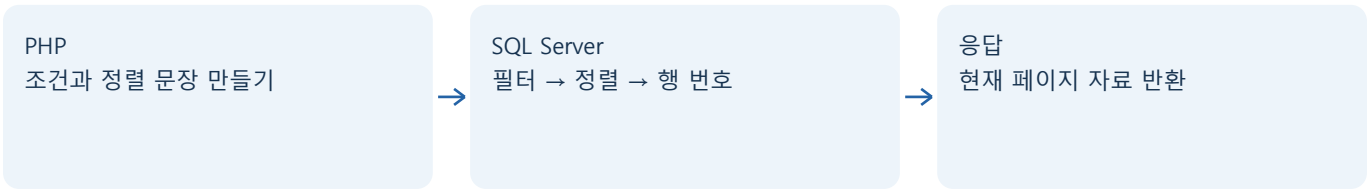
짧은 소스가 많은 일을 하는 이유

먼저 확인할 사실: 다섯 가지 요구와 현재 실행 코드의 차이

이 파일은 PHP가 SQL 문장을 만들고, SQL Server가 검색·정렬·번호 매기기를 실행하는 구조입니다. PHP에 직접 정렬 알고리즘을 작성하지 않아도 ORDER BY에 우선순위를 적으면 DB가 전체 자료를 비교합니다.



요청	현재 기본 정렬에서의 실제 상태
1. 등록일 2025-09-01 전후	미적용. 기준 변수는 있으나 사용되지 않고, 관련 CASE는 주석 안에 있습니다.
2. 계약일자 없는 자료 우선	적용. NULL·빈 문자열·일반 공백만·공백 제거 후 --인 자료가 앞 그룹입니다.
3. 연월만 입력한 자료 구분	미적용. 길이로 구분하는 CASE는 예전 주석 코드에만 있습니다.
4. 계약일자 최신순	적용. 계약일자 있는 그룹에서 ContractDate DESC입니다.
5. 수정일자 최신순	조건부 적용. 앞 정렬값이 같은 자료끼리 EditDate DESC입니다. 전체 자료를 수정일만으로 정렬하지 않습니다.

위 판정은 열 제목 정렬이 비어 있고, SortType 1~9가 기본 정렬을 덮어쓰지 않는 경우입니다. 메뉴 정렬 또는 열 제목 클릭을 하면 다른 ORDER BY가 적용됩니다.

소스를 읽을 때 먼저 구분할 것

/* ... */ 안은 실행되지 않는 설명·옛 코드입니다. // 뒤도 해당 줄의 주석입니다. 주석에 요구가 적혀 있어도 구현된 기능이라는 뜻은 아닙니다. 이 문서는 주석의 문장을 지시로 따르지 않고, 실제 실행되는 코드를 기준으로 설명합니다.

확인 위치: 기준 변수 265행, 기본 정렬 332~346행, 정렬 대체 349~379행. DB 자료형·실제 저장값·포함 파일 setHeader.php 내용은 제공되지 않아 런타임 검증은 하지 않았습니다.

정렬은 네 개의 비교 열쇠

ORDER BY는 왼쪽 기준부터 비교하고, 같을 때만 다음 기준을 사용합니다

```
ORDER BY
  CASE WHEN ContractDate IS NULL
    OR LTRIM(RTRIM(ContractDate)) = ''
    OR REPLACE(ContractDate, ' ', '') = '--'
    THEN 0 ELSE 1 END ASC,
  CASE WHEN ContractDate IS NULL
    OR LTRIM(RTRIM(ContractDate)) = ''
    OR REPLACE(ContractDate, ' ', '') = '--'
    THEN NULL ELSE ContractDate END DESC,
  a.EditDate DESC,
  a.PK_ID DESC
```

① 그룹
없음=0
있음=1
작은 값 먼저

→

② 계약일
큰 값 먼저
최신 계약 우선

→

③ 수정일
앞 두 값이 같을 때
최신 수정 우선

→

④ 고유번호
앞 세 값이 같을 때
큰 PK 먼저

CASE는 자료를 바꾸지 않고, 비교용 값을 만듭니다

첫 CASE는 각 행에 0 또는 1이라는 가상의 정렬표를 붙입니다. ASC는 작은 값부터이므로 0(미입력)이 1(입력됨)보다 앞입니다. 화면에 별도 그룹 칸을 표시하는 코드는 아닙니다.

둘째 CASE는 미입력 자료의 비교값을 모두 NULL로 통일합니다. 따라서 NULL·빈 문자열·공백·---처럼 저장 모양이 달라도 미입력 그룹 안에서는 계약일 비교값이 같아지고, 다음 기준인 수정일로 순서를 결정합니다.

예를 들어 계약일이 2026-10-08인 자료는 계약일이 2026-10-07인 자료보다 먼저입니다. 뒤 자료의 수정일이 더 최신이어도 계약일 우선순위를 뒤집지 못합니다.

표현	읽는 방법
ASC / DESC	작은 값부터 / 큰 값부터
LTRIM(RTRIM(x))	양끝의 일반 공백을 제거
REPLACE(x, ' ', '')	일반 공백을 모두 제거. ' - - ' → '--'
a.EditDate / a.PK_ID	a는 table_contract의 별명. 해당 계약서의 수정일 / 고유번호

실제로 줄을 세워 보면

예시로 보는 기본 정렬과, 실행되지 않는 옛 정렬

순서·자료	등록일	계약일	수정일
1 · A	2024-01-10	NULL	2026-10-09
2 · B	2026-10-01	빈 문자열	2026-10-08
3 · C	2026-10-01	2026-10-08	2026-10-09
4 · D	2026-10-02	2026-10-08	2026-10-07
5 · E	2026-10-03	2026-10-07	2026-10-09
6 · F	2026-10-04	2026-10	2026-10-09

A는 기준일 이전에 등록됐지만 계약일이 없어서 맨 앞 그룹입니다. C와 D는 계약일이 같으므로 수정일로 비교합니다. E의 수정일은 D보다 최신이지만 계약일이 더 오래되어 뒤입니다. F의 위치는 문자열 YYYY-MM 형식으로 저장됐다는 가정의 예시입니다.

2025-09-01과 연월 구분은 어디에 있나요?

```
$BaseMaxDate = '2025-09-01';
/*
  CASE WHEN RegDate >= '2025-09-01' ... END ASC,
  CONVERT(VARCHAR(7), RegDate, 23) DESC,
  CASE WHEN Len(LTrim(RTrim(ContractDate))) < 10
       THEN 0 ELSE 1 END ASC,
  ...
*/
```

265행의 변수 선언만으로 자료를 구분하지는 않습니다. 290~325행의 옛 정렬은 기준일 이후 미입력=0, 기준일 이후 입력됨=1, 기준일 이전=2로 나누고, 그 안에서 등록 연월을 비교합니다. 그 다음에 짧은 계약일 문자열을 구분합니다. 이 전체 구간이 주석이라 현재 효과가 없습니다.

길이가 10 미만이라는 검사는 연월 날짜인지 정확히 검증하는 방법도 아닙니다. 예를 들어 잘못 입력한 짧은 문자열도 같은 그룹이 될 수 있습니다. YYYY-MM-DD가 일정한 폭으로 저장되면 문자열 최신순과 날짜 최신순이 맞지만, YYYY-M-D나 혼합 형식은 달라질 수 있습니다.

다섯 요구를 다시 적용하려면 먼저 정할 순서

"등록일 전후 그룹이 먼저"라면 기준일 이전의 미입력은 기준일 이후의 입력된 자료보다 뒤가 될 수 있습니다. "모든 미입력 자료가 먼저"라면 미입력 여부를 등록일 구분보다 앞에 둬야 합니다. 두 우선순위는 결과가 다르므로 요구 번호만으로 같은 정렬이라고 판단할 수 없습니다.

검색 소스의 전체 지도

검색은 어디를 찾을지 선택하고, 같은 비교식을 반복 생성하는 과정입니다



단계	코드의 역할	예: 이름 검색
1 · 검색어 준비	작은따옴표 처리, 공백 제거 버전 생성	홍 길 동 → 홍길동
2 · switch	검색 종류별로 필드 목록 선택	SearchType=3 → 이름 필드
3 · foreach	목록의 각 필드에 같은 비교식 적용	L_Name 비교, H_Name 비교...
4 · implode	조건 사이에 OR를 넣어 연결	이름1 일치 OR 이름2 일치
5 · EXISTS	연결된 별지 중 일치하는 행 존재 확인	별지 이름에서라도 찾으면 참
6 · .=	기존 필터 뒤에 통합 조건 덧붙임	MainID 등 AND (본문 OR 별지)

PHP 문법을 익숙한 작업으로 읽기

문법	뜻 / Delphi와 비교
array(...)	검색할 필드를 담는 목록. TStringList 항목 목록과 비슷합니다.
$items[] = ...	목록 끝에 하나 추가. TStringList.Add와 비슷합니다.
foreach ($fields as $field)	목록에서 필드를 하나씩 꺼내 반복합니다.
implode(' OR ', $conditions)	여러 문자열 사이에 OR를 끼워 하나로 합칩니다.
. / .=	문자열 연결 / 기존 문자열 뒤에 덧붙임. Delphi의 +와 비슷합니다.
switch / case / break	종류 번호로 분기 / 해당 항목 / 분기 종료입니다.

이 코드는 switch에서 DB를 조회하지 않습니다. 필드 목록을 먼저 고르고, 공통 반복문이 SQL 텍스트를 만듭니다. 실제 조회는 뒤의 sqlsrv_query에서 실행됩니다. 같은 LIKE 문장을 종류별로 일일이 쓰지 않아서 코드가 짧아집니다.

검색 종류별로 어디를 찾나요?

a = 계약서 본문 / b = 별지. 두 목록은 서로 다른 테이블의 검색 대상입니다

종류	본문 mainFields	별지 beljiFields / Gubun
0 부동산 주소	Address	J_Address, J_Si, J_Gu, J_Dong, J_JiBun / 004
1 동수	DongNo, Address, L/H_Address, L/H_AgentAddress	J_Address, J_Dong / 001·002·004
2 호수	HosuNo, Address, L/H_Address, L/H_AgentAddress	J_Address / 001·002·004
3 고객 이름	L_Name, H_Name, L_AgentName, H_AgentName, YoungSuin	J_Name, j_Name_2 / 001·002
4 고객 전화	L/H_Phone_E 및 L/H_AgentPhone_E를 복호화한 값	J_Tel_E, J_Phone_E 복호화값 + J_Tel, J_Phone / 001·002·003
5 고객 주소	L/H_Address 및 L/H_AgentAddress	J_Address, J_Si, J_Gu, J_Dong / 001·002
6 중개업소명	OfficeName, OfficeName1	J_SangHo, J_Name / 003

표의 L/H는 양쪽 필드를 줄여 쓴 표시입니다. 예: L/H_Address는 L_Address와 H_Address입니다. Gubun의 의미는 첨부 코드 주석 기준으로 001·002=계약 당사자, 003=중개업소, 004=부동산입니다. 실제 DB 정의는 별도 확인이 필요합니다.

동수·호수 검색 범위가 넓은 이유

동수 검색은 DongNo만 검사하지 않습니다. 별지 또는 고객 주소에 "905동 2205호"처럼 적힌 자료까지 찾기 위해 여러 주소 필드를 함께 검사합니다. 반대로 검색어 "905"가 다른 주소 숫자에 들어 있어도 포함될 수 있습니다. 동 번호를 정확히 해석하는 검색이 아니라 문자열 포함 검색입니다.

별지의 주소 조각은 각각 비교합니다

J_Si, J_Gu, J_Dong을 하나의 주소로 이어 붙여 검색하지 않습니다. 각 칸을 따로 OR로 비교합니다. "서울강남"이 두 칸에 나뉘어 저장됐고 J_Address에도 전체 주소가 없다면, 이 코드에서는 찾지 못할 수 있습니다.

검색어가 비어 있으면 검색 조건을 추가하지 않습니다. 숫자로 변환한 SearchType이 0~6 밖이면 mainFields가 비어 있어 이 통합 검색 조건이 추가되지 않습니다. 숫자가 아닌 입력은 (int) 변환 시 0으로 바뀔 수 있으므로 입력 검증과는 다른 동작입니다.

“홍 길 동”을 검색하는 과정

검색어 변환 → 비교식 생성 → 본문 또는 별지에서 일치

사용자 입력
SearchType = 3
홍 길 동

→

원문 검색어
SearchText
홍 길 동

→

공백 제거
SearchText_2
홍길동

```
$SearchText = str_replace("'", "''", $SearchText);
$SearchText_2 = preg_replace('/₩s+/u', '', $SearchText);

(ISNULL(a.L_Name,'') LIKE '%홍 길 동%'
 OR REPLACE(ISNULL(a.L_Name,''),' ','') LIKE '%홍길동%')
```

ISNULL은 NULL 값을 빈 문자열로 바꿉니다. LIKE는 패턴과 비교하며 %는 앞뒤에 어떤 글자가 있어도 된다는 뜻입니다. 첫 비교는 원문 검색어로 찾고, 둘째 비교는 DB 칸의 일반 공백을 제거한 값에서 공백 없는 검색어를 찾습니다.

저장 이름	비교 결과	이유
홍길동	일치	둘째 비교에서 홍길동 포함
홍 길 동	일치	첫째 비교 또는 공백 제거 후 일치
김홍길동	일치	%홍길동%는 부분 포함도 허용
홍[탭]길동	보장되지 않음	DB 값에서는 일반 공백만 지우므로 탭은 남음

전화번호는 별도 규칙입니다

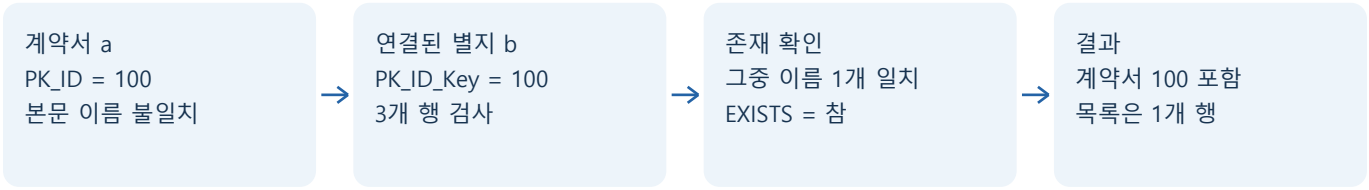
```
010-1234-5678 → 검색어 01012345678
REPLACE(REPLACE(ISNULL(복호화한전화,''),'-',''),' ','')
  LIKE '%01012345678%'
```

SearchPhone은 입력에서 하이픈·일반 공백·탭·줄바꿈을 제거합니다. DB 전화값에서는 하이픈과 일반 공백만 제거합니다. 암호화 필드는 SQL Server 함수 dbo.fn_DecryptByPass를 호출해 비교합니다. PHP가 직접 복호화하는 구조는 아닙니다.

작은따옴표는 O'Brien → O''Brien처럼 SQL 문자열 안에서 표현하도록 바꿉니다. %·_·[같은 LIKE 패턴 문자는 이 코드에서 리터럴 문자로 처리하지 않습니다. 입력 변환과 DB 값 변환의 범위가 완전히 같지도 않습니다.

EXISTS: 별지 중 하나라도 맞나요?

한 계약서에 별지가 여러 장이어도, 계약서 목록 행은 한 번만 나옵니다



계약서	별지	이름 검색 결과
100	행 1 / Gubun 001 / 김철수	불일치
100	행 2 / Gubun 002 / 홍길동	일치
100	행 3 / Gubun 002 / 홍 길 동	일치

```
EXISTS (
  SELECT 1
  FROM table_contract_belji b WITH(NOLOCK)
  WHERE LTRIM(RTRIM(b.PK_ID_Key))
        = CONVERT(VARCHAR(30), a.PK_ID)
    AND b.Gubun IN ('001','002')
    AND (별지이름조건1 OR 별지이름조건2)
)
```

이 SQL은 현재 계약서의 PK_ID와 연결되는 별지만 확인합니다. 예를 들어 a.PK_ID가 숫자 100이면 문자열 "100"으로 변환하고, 별지의 PK_ID_Key 양끝 공백을 제거한 값과 비교합니다. 실제 연결 키가 정확하다는 전제가 필요합니다.

SELECT 1의 1은 검색 건수나 계약서 번호가 아닙니다. EXISTS는 조건에 맞는 행이 존재하는지를 판단합니다. 논리적으로 하나 이상이면 참이며, 실제 실행 계획과 검사 방법은 DB가 결정합니다.

단순 JOIN과 결과가 다른 이유

일치하는 별지를 단순 JOIN하면 예시에서 계약서 100이 별지 행 2·3과 각각 결합되어 두 행으로 나올 수 있습니다. EXISTS는 참/거짓만 바깥 계약서 행에 전달하므로 이 검색 때문에 계약서 행이 늘어나지 않습니다. 그래서 count(*)에도 같은 조건을 재사용하기 좋습니다.

EXISTS가 막는 것은 별지 검색으로 인한 중복입니다. 목록에서 쓰는 ArticleTypeMenu 등의 다른 JOIN이 일대일인지까지 보장하는 것은 아닙니다.

최종 SQL은 이렇게 조립됩니다

WHERE는 남길 자료, ORDER BY는 순서, ROW_NUMBER는 페이지용 번호입니다

```
WHERE a.MainID = '예시업소'
  AND (기존의 매물·거래·기간 등 조건)
  AND (
    (본문이름조건1 OR 본문이름조건2 OR ...)
    OR EXISTS (연결된 별지에서 이름이 맞는 행)
  )
```

바깥 AND는 기존 조건과 검색 조건을 모두 만족해야 한다는 뜻입니다. 안쪽 OR는 본문 또는 별지 중 어느 한쪽에서만 찾아도 된다는 뜻입니다. 괄호가 없으면 다른 업소나 다른 필터의 자료까지 포함될 수 있어, 이 구조에서 괄호가 핵심입니다.

① 필터
WHERE
해당 자료만 남김

→

② 줄 세우기
ORDER BY
정렬 기준 비교

→

③ 번호
ROW_NUMBER
1, 2, 3...

→

④ 페이지
BETWEEN
필요한 번호만

```
SELECT * FROM (
  SELECT ROW_NUMBER() OVER (정렬문장) AS RowNum,
         조회필드들
  FROM table_contract a ...
  WHERE 필터문장
) AS X
WHERE RowNum BETWEEN 36 AND 70
```

PageRow는 요청값을 읽은 뒤 35로 고정합니다. 1페이지는 1~35번, 2페이지는 36~70번입니다. 끝 번호가 총 건수보다 크면 총 건수까지 줄입니다. 별도 count(*) 조회도 같은 $strFilter를 사용합니다.

변수·호출	역할
$Field	화면에 보낼 열과 FROM/JOIN을 담습니다.
$strFilter	업소·분류·기간·검색 등의 WHERE 조건입니다.
$strSort	ORDER BY 정렬 문자열입니다.
sqlsrv_query / fetch_object	DB에서 실행 / 결과를 한 행씩 가져옵니다.
$Response / printfView	총 건수와 자료를 담아 응답을 출력합니다.

주의: 바깥 SELECT에 ORDER BY RowNum이 없으므로 SQL 규칙상 최종 전달 순서까지 보장하지는 않습니다. 정렬에 따른 행 번호와 페이지 소속은 결정되지만, 반환 표시 순서를 확실히 하려면 바깥 ORDER BY RowNum도 필요합니다.

다른 정렬이 기본 정렬을 대체합니다

현재 동작을 오해하기 쉬운 부분과 소스에서 찾는 위치

입력 상태	실행되는 정렬
열 제목 필드가 비어 있고 SortType 1~9 아님	미입력 우선 → 계약일 DESC → 수정일 DESC → PK DESC
열 제목 필드가 비어 있고 SortType=1	EDITDATE + 요청 방향 → PK DESC
열 제목 필드가 비어 있고 SortType=2~9	각 선택 필드 + 요청 방향 → PK DESC
열 제목 필드가 있음	클릭 필드 + 클릭 방향 → PK도 같은 방향

SortType 2~9는 순서대로 AptNameTitle, TradeClass, L_Name, H_Name, ExpireDate, DongNo, Address, RemainderDate입니다. 기본 정렬 뒤에 추가하는 방식이 아니라 $strSort 자체를 다시 대입하므로 기본 미입력 우선 규칙도 사라집니다.

소스의 위치를 따라 읽는 순서

행	찾아볼 내용
32~77	JSON 요청에서 검색·정렬·페이지 값 받기
114~247	MainID 등 기존 필터 추가
265~346	기준일 변수, 주석 속 옛 정렬, 현재 기본 정렬
349~379	정렬 메뉴 및 열 제목 클릭에 따른 대체
465~538	검색어 준비 및 검색 대상 선택
544~585	반복 비교식, EXISTS, 최종 검색 필터 조립
608~665	전체 건수, 페이지 번호, 최종 조회, 응답 자료

실제 코드에서 확인된 보완 지점

① 검색어의 따옴표 처리만으로 모든 입력이 안전해지지는 않습니다. MainID·날짜·정렬 필드 등이 문자열로 결합됩니다. 값은 바인딩하고 정렬 필드·방향은 허용 목록으로 제한하는 편이 좋습니다.
② WITH(NOLOCK)은 미커밋 자료 등을 읽을 수 있어 조회 중 목록과 건수가 어긋날 수 있습니다.
③ 공백 제거·복호화·앞쪽 % 검색은 자료량이 많을 때 검색 비용이 커질 수 있습니다. 실제 성능은 실행 계획 확인이 필요합니다.

④ 표시용 ContractDate는 SELECT에서 "입력안함"으로 가공하지만, 정렬 CASE는 조회 단계의 원본 날짜값을 기준으로 합니다. 표시용 미입력 판정은 정렬 판정과 완전히 같지 않아서 공백만 저장된 값 등의 화면 표시는 별도 확인이 필요합니다.
⑤ MouseScrollAction은 608행에서 0으로 고정되어 매번 건수 조회 분기를 실행합니다.

이 문서는 첨부 파일의 실행·주석 구간을 읽어 만든 해설입니다. DB 접속 및 PHP 실행을 통한 결과·성능 검증은 포함하지 않습니다. 원본 파일과 프로젝트 앱 소스는 수정하지 않았습니다.